Backend-ul, feature cu feature

Spring Boot 3.5 · Java 21 · Maven · MySQL 8.4 · Flyway · Docker Hub · GitHub Actions · EC2 · Sebi

Ținta e în `solar-sync-broker-docs/`: `STRUCTURA_BD.pdf` = tabelele, `FEATURES.pdf` = ce trebuie să meargă și cum merge azi, `openapi.yaml` = contractul (îl deschizi în `editor.swagger.io`; metodele din controllere = `operationId`). Pornești de la codul tău din `ab630df`, nu de la zero.
Fiecare feature trece prin același drum:
migrare `Vn__…sql` (dacă schimbi tabele) → repository → service → controller → test de integrare, cu login real → push → verde în Actions → verificat pe EC2

0 Întâi: ce e de reparat în codul de azi

- `HouseController` cere `house:view` / `house:manage`, dar token-ul conține `HOUSE_VIEW` / `HOUSE_MANAGE` → 403 pentru toată lumea. Plus un test de integrare pentru case (azi nu există).
- `updateConfiguration` are `@Transactional(readOnly = true)`: răspunde 200 și nu salvează nimic. `@Transactional` simplu; testul cere valorile noi.
- `executeCommand` pune `targetSoc` pe poziția lui `powerKw` din `BatteryCommand`: un procent ajunge în motor ca kW. Câmpul devine `powerKw` (DTO, entitate, coloană), cu `@Positive`.
- Serviciile aruncă `HouseAccessDeniedHandler`, pe care nu-l prinde niciun `@ExceptionHandler` → 500. O singură excepție de acces, pe 403; handler-ele întorc `ApiErrorResponse` întreg (azi trimit doar `error`, adică `null` → corp gol).
- Cele 3 clase `*IT` nu rulează la `mvn test`: `maven-failsafe-plugin` + `mvn verify`.
- Nicio cale nu creează o baterie: `createHouse` o creează odată cu casa, cu valorile implicite din `FEATURES.pdf`.
- Proprietarul n-are `BATTERY_COMMAND` și `BATTERY_CONFIG` → 403 pe propria baterie. `register` ia `userType` de la client, deci oricine devine `ADMIN`. `updateConfiguration` nu verifică al cui e bateria.

Gata când: `mvn verify` verde, iar cu login real: ADMIN → `GET /api/v1/houses` = 200; proprietarul → `CHARGE` 2 kW = 201 `SUCCESS`, SoC 50 → 69.

1 Conturi — și tot drumul până pe EC2

a) Repo-ul după restructurare

- Proiectul Maven stă în `solar-sync-broker-api/`. În `solar-sync-broker-ops/`: `Dockerfile`, `docker-compose-local.yaml` și `docker-compose-platform.yaml`, compose-ul de deploy al solar-sync (MySQL + aplicația). Workflow-ul rămâne în `.github/workflows/build-push.yaml`, la rădăcină.
- `paths`: `solar-sync-broker-api/**`, `solar-sync-broker-ops/Dockerfile` și workflow-ul — un push doar în docs nu mai pornește build-ul. `test` rulează în `solar-sync-broker-api`; după failsafe, `mvn -B verify` în loc de `test`. `build-push`: `context: solar-sync-broker-api` + `file: solar-sync-broker-ops/Dockerfile`.
- Joburile: `test` (37 de teste verzi) → `build-push` (imaginea `sebi123q/solarsyncbroker-app:sha-…` în Docker Hub) → `update-compose` (tag-ul în `docker-compose-local.yaml`) → `update-platform`, care scrie tag-ul în `docker-compose-platform.yaml`, din același repo, fără niciun secret în plus:

```
update-platform:
  needs: [build-push, update-compose]
  permissions:
    contents: write
  env:
    TAG: ${{ needs.build-push.outputs.tag }}
  steps:
    - uses: actions/checkout@v7
    - run: |
        git pull --rebase
        grep -q "image: ${IMAGE}:" "${COMPOSE_PLATFORM}"
        sed -i "s|image: ${IMAGE}:.*|image: ${IMAGE}:${TAG}|" \
          "${COMPOSE_PLATFORM}"
    - run: |
        git add "${COMPOSE_PLATFORM}"
        git commit -m "platform: ${IMAGE}:${TAG}"
        git pull --rebase && git push
```

b) Flyway: V1 = schema de acum

- Dependențele de Flyway le ai deja: `spring.flyway.enabled: true` și `ddl-auto: validate`. `validate` verifică **toate** entitățile la pornire, deci `V1__schema.sql` (în `db/migration/`, azi gol) creează toate cele 6 tabele de azi din `STRUCTURA_BD.pdf`, deja corectate.
- Baza `solar_db` din `docker-compose-platform.yaml` e creată azi de `ddl-auto`: o ștergi o singură dată (datele sunt generate), altfel Flyway refuză o schemă care nu e goală.
- Testele pe MySQL din Testcontainers (dependențele sunt deja în `pom.xml`), cu `@ServiceConnection`: rulează același V1. H2 iese.
- De aici, orice schimbare de schemă = un `Vn` nou; V1 nu se mai editează după ce a rulat pe EC2.

c) Codul

- Permisiunile: o singură hartă `UserType` → permisiuni, citită de `getAuthorities()`; tabelul `user_permissions` iese din V1. Enum-ul primește tot ce cer feature-urile: `TELEMETRY_VIEW`, `COMMUNITY_VIEW`, `FINANCIAL_VIEW`, `GRID_VIEW`, `SIMULATION_CONTROL`, `AGENT_VIEW`, `AGENT_OPERATE`, `USER_MANAGE`; `BATTERY_MANAGE` pleacă.
- `register` face doar `USER`. Un `ApplicationRunner` creează contul `ADMIN` din `ADMIN_EMAIL` / `ADMIN_PASSWORD`, dacă nu există. Acestea și `JWT_SECRET_KEY` stau în `.env` pe EC2 — niciodată în repo.
- `getCurrentUser` pe `GET /api/v1/auth/me`. `addUser` se mută pe `POST /api/v1/users`, cu `USER_MANAGE`, și salvează parola prin `PasswordEncoder` și tipul.
- Testul de integrare **prin login real**, nu cu `@WithMockUser` (ăla își inventează singur permisiunile, deci nu vede un `hasAuthority` pe care login-ul nu-l dă niciodată): register → login → `/auth/me` = 200; fără token = 401; `addUser` ca `USER` = 403.

> **Gata când:** un push → toate 4 joburile verzi → `docker-compose-platform.yaml` are tag-ul nou → pe EC2 (deploy-ul automat e pasul următor) login + `/api/v1/auth/me` = 200.

2 Case

- Cele 6 operații din `openapi.yaml`. `createHouse` salvează casa și bateria într-o tranzacție. Ștergerea unei case cu istoric → 409.
- Proprietarul se verifică în SQL, cu derived queries: `existsByIdAndOwner_Email`, `findByOwner_Email`. `ADMIN` și `OPERATOR` ocolesc verificarea, scrisă într-o singură metodă.

> **Gata când:** pe EC2: ADMIN creează casa lui X → X o vede, cu bateria ei; Y primește 403, nu 500.

3 Bateria casei

- `getBattery`, `updateConfiguration` (cu verificarea de proprietar), `sendCommand`, `getCommands`.
- `catch (Exception)` devine `catch` doar pe cele două excepții ale validatorului, iar numele lor ajunge în `failure_reason`. Capacitatea vine din baterie; 10 și 90 stau într-un singur loc, nu în serviciu și în validator.

> **Gata când:** `CHARGE` 90 kW → 201 `REJECTED` `BATTERY_POWER_LIMIT`, SoC neschimbat; istoricul îl vede proprietarul.

4 Simulare și telemetrie

- `DemoDataSeeder` cu `@Profile("demo")`: 10 case cu puteri PV diferite; iese dacă există deja case.
- `PvGenerator`, `LoadGenerator` și `PriceGenerator`: calcul pur, ca `EnergyEngineImpl`, cu sămânța din `SimulationProperties`; teste unitare.
- `SimulationService.tick()`: pentru fiecare casă activă PV + consum → `EnergyEngine.compute` → `Telemetry`, un singur `saveAll`. `TelemetryRepository` decomentat, cu `findTop50ByHouseIdOrderByRecordedAtDesc`.

> **Gata când:** 10 tick-uri → 100 de rânduri; aceeași sămânță → aceleași valori la a doua rulare.

5 Financiar

- Tick-ul cheamă `recordTransaction`: import → `BUY`, export → `SELL`, la prețul tick-ului; `BigDecimal` pe bani.
- `FinancialController` pe `/api/v1`: `/houses/{houseId}/financial-summary` cu ownership, `/community/financial-summary`.

> **Gata când:** după un tick, suma `total_value` la `BUY` pe o casă = importul ei × prețul din telemetrie.

6 Rețeaua

- `V2__grid_snapshots.sql`. `GridGuard`: calcul pur, din importul și exportul total + `GridProperties` → stare și kW în plus; teste pe cele trei praguri. Tick-ul salvează un rând.

> **Gata când:** o comunitate încărcată artificial peste 50 kW → `getGridState` = `CRITICAL`.

7 Agentul — aici e lucrarea

- `V3__agent.sql`: `agent_runs`, `agent_decisions` și `command_executions.agent_decision_id`.
- `AgentPlanner`: calcul pur, câte o metodă pe regulă (în lucrare, fiecare devine un paragraf). Fiecare decizie trece prin `BatteryCommandValidator`; în `ADVISORY` doar se salvează. `approveAgentRun` refolosește calea din `sendCommand`, nu scrie una nouă.

> **Gata când:** pe EC2: tick → `runAgent` → `approveAgentRun` → comenzi cu `agent_decision_id` și SoC schimbat. Ciclul OBSERVE → PLAN → VALIDATE → ACT e complet.

8 Opțional, după 7

- `@Scheduled` peste `tick()` și agent (tick-ul manual rămâne, pentru teste). Apoi un model AI propune planul (`spring-ai-bom` e deja în `pom.xml`), prin aceeași poartă: `BatteryCommandValidator`.

> **Rămâi pe Spring Boot 3.5 până la susținere.** Exemplele din proiectele pe Boot 4 nu se copiază 1:1: la tine `@MockBean` e încă valid, `springdoc-openapi` 2.8.x e versiunea corectă, Jackson e `com.fasterxml.jackson`, iar `@WebMvcTest` vine din `org.springframework.boot.test.autoconfigure.web.servlet`.